ISCD Worksheet 5.ii

Anurag Bhunia(bmat2508@isibang.ac.in)

1.

```
set.seed(100)
n=10000
A=rpois(n,6)
B=rpois(n,3)
C=rpois(n,3)
d=B^2 - 4*A*C
p=length(d[d>=0])/n
p
```

```
## [1] 0.1076
```

2. a)

i)

```
x=c(1,2,3,4)
cumsum(x)
```

```
## [1]  1  3  6 10
```

This function finds the cumulative sum of the entries in the vector and creates a vector out of them.

ii)

```
runningmean=function(x,N){
    y=sample(x,N,replace=T)
    c=cumsum(y)
    n=1:N
    c/n
}
runningmean(c(1,2,3,4,5),10)
```

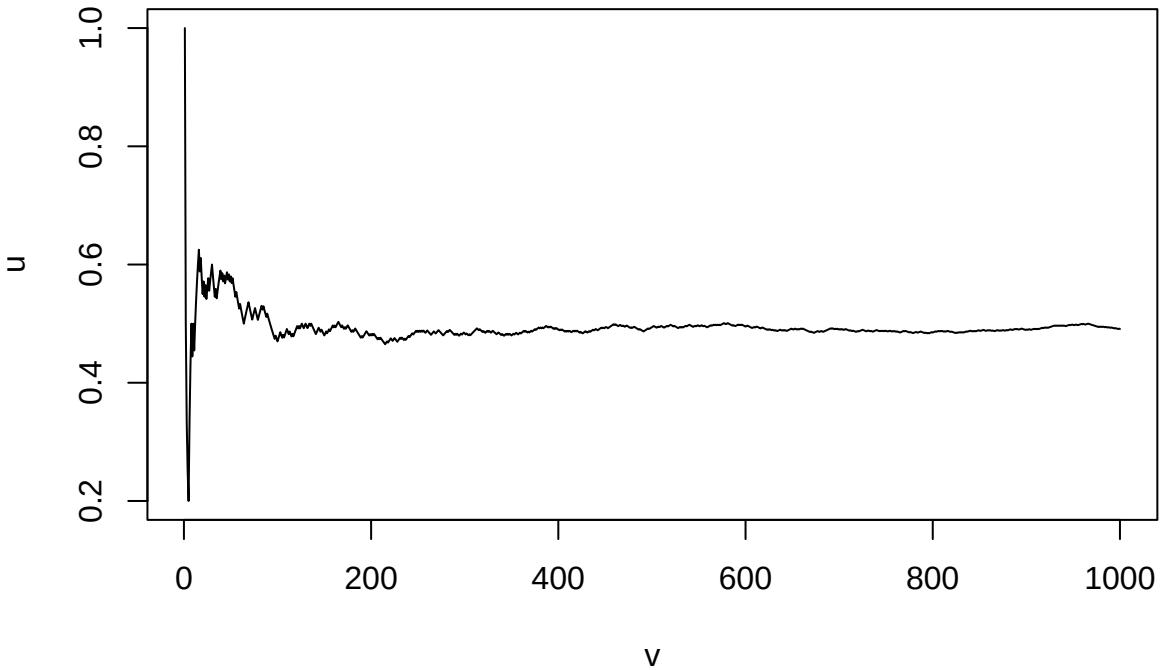
```
##  [1] 4.000000 4.000000 3.000000 3.000000 3.200000 3.333333 3.571429 3.250000
##  [9] 3.000000 2.800000
```

The runningmean function is taking N samples from a vector x with replacement and is thus creating a sequence y1,y2,...yN. Then it calculates the cumulative mean of this vector sequence ie. mean of the first term, mean of first two terms, and so on till the mean of the whole vector.

iii)

```
runningmean=function(x,N){
    y=sample(x,N,replace=T)
    c=cumsum(y)
    n=1:N
    c/n
}
```

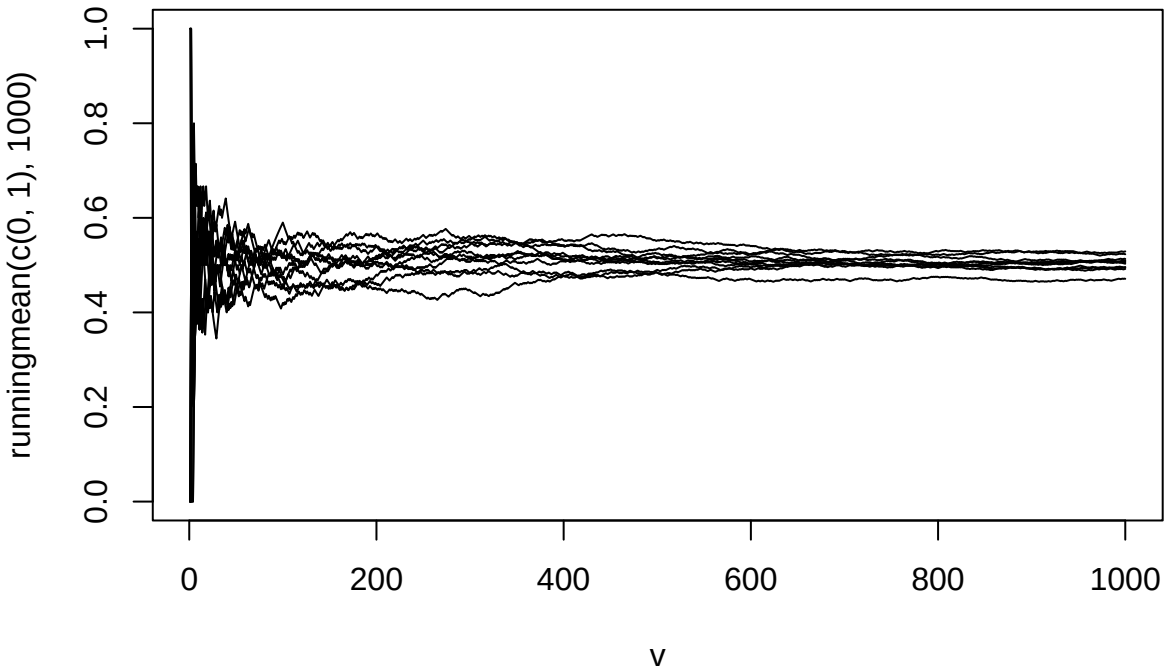
```r
u=runningmean(c(0,1),1000)
v=1:1000; plot(u~v,type="l");
```



This plot shows that as the number of cumulative means increases, its value, though fluctuating a lot at the start, eventually stabilizes to a value quite close to 0.5.

iv)

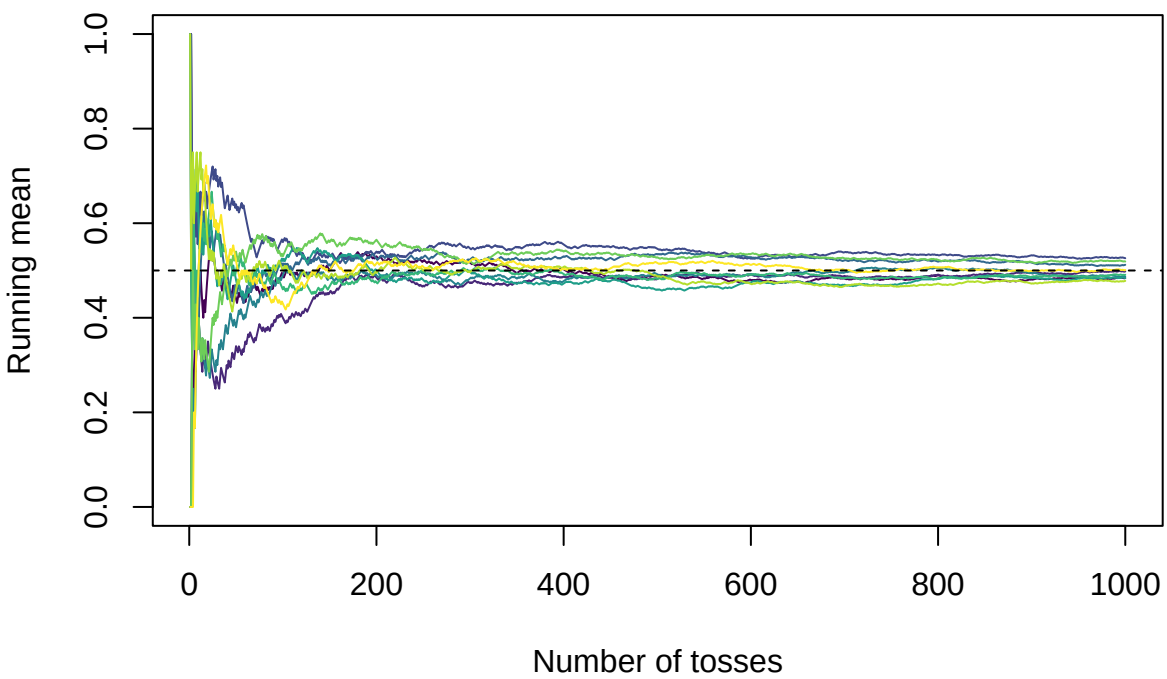
```r
runningmean=function(x,N){
    y=sample(x,N,replace=T)
    c=cumsum(y)
    n=1:N
    c/n
}
v=1:1000
plot(runningmean(c(0,1),1000) ~ v,type = "l", ylim = c(0,1))
replicate(10,lines(runningmean(c(0,1),1000)~v,type="l"))
```




```
## [[1]]
## NULL
##
## [[2]]
## NULL
##
## [[3]]
## NULL
##
## [[4]]
## NULL
##
## [[5]]
## NULL
##
## [[6]]
## NULL
##
## [[7]]
## NULL
##
## [[8]]
## NULL
##
## [[9]]
## NULL
##
## [[10]]
## NULL
```

v)

```r
library(viridisLite)
runningmean = function(x, N){
  y = sample(x, N, replace = TRUE)
  c = cumsum(y)
  n = 1:N
  c / n
}
v = 1:1000
cols = viridis(10)
plot(runningmean(c(0,1), 1000) ~ v,
     type = "l",
     col = cols[1],
     ylim = c(0,1),
     xlab = "Number of tosses",
     ylab = "Running mean")
for(i in 2:10){
  lines(runningmean(c(0,1), 1000) ~ v, col = cols[i])
}
abline(h = 0.5, lty = 2)
```



b)

c)

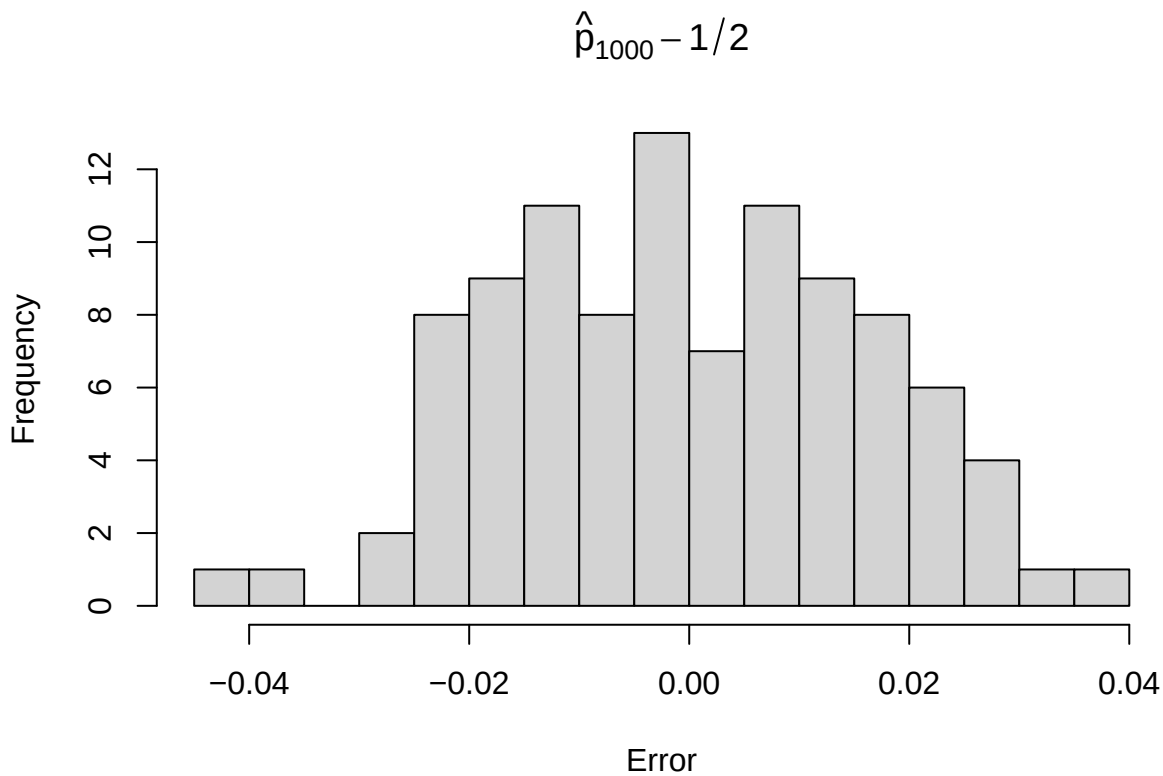
```r
set.seed(1)
p_hat = replicate(100, {
  x = rbinom(1000, 1, 0.5)
  mean(x)
})
p_hat
```

```
##   [1] 0.480 0.481 0.481 0.492 0.502 0.522 0.526 0.478 0.498 0.493 0.524 0.483
##  [13] 0.487 0.507 0.498 0.513 0.457 0.515 0.521 0.501 0.484 0.502 0.483 0.493
##  [25] 0.482 0.491 0.499 0.517 0.475 0.494 0.486 0.521 0.507 0.517 0.514 0.506
```

```
##  [37] 0.494 0.513 0.478 0.499 0.538 0.515 0.523 0.507 0.482 0.489 0.504 0.518
##  [49] 0.520 0.506 0.534 0.487 0.480 0.507 0.501 0.516 0.517 0.515 0.496 0.480
##  [61] 0.521 0.510 0.514 0.520 0.481 0.497 0.507 0.503 0.506 0.473 0.506 0.530
##  [73] 0.512 0.487 0.527 0.489 0.497 0.496 0.496 0.495 0.487 0.478 0.478 0.489
##  [85] 0.514 0.496 0.491 0.506 0.488 0.498 0.503 0.465 0.500 0.519 0.482 0.530
##  [97] 0.480 0.496 0.488 0.488
```

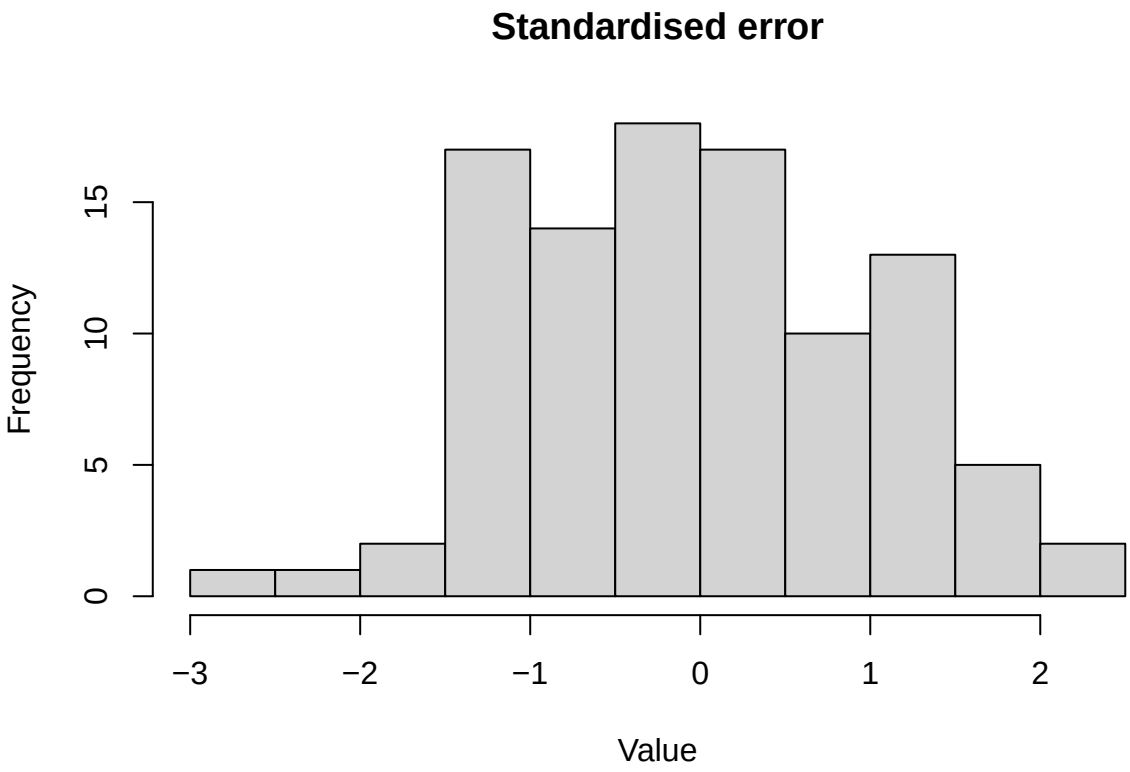
ii)

```r
error = p_hat - 0.5
hist(error,
     breaks = 15,
     main = expression(hat(p)[1000] - 1/2),
     xlab = "Error")
```



iii)

```r
std_error = (p_hat - 0.5) / (1 / (2 * sqrt(1000)))
hist(std_error,
     breaks = 15,
     main = "Standardised error",
     xlab = "Value")
```



iv) The standardised error seems to follow approximately a normal distribution, as is seen from the graph. This is my guess.